

Perfecto, este es un proyecto muy bien planteado — la clave está en que el operador nunca tenga que "pensar en ISA-5.1", solo elegir de listas y confirmar. Vamos paso a paso.

1. Stack recomendado

Python 3 + SQLite + Tkinter (interfaz de escritorio nativa)

Opción	Por qué la recomiendo / descarto
✔ Python + Tkinter + SQLite	Viene incluido en Python (sin instalar nada extra), no abre puertos de red ni servidores — importante en el entorno IT/OT de una planta donde correr un servidor local (Flask) puede chocar con políticas de seguridad. Se puede empaquetar como un único <code>.exe</code> con PyInstaller para repartirlo por USB, sin que el usuario necesite tener Python instalado.
⚠ Python + Flask + SQLite (web local)	Interfaz más bonita (HTML/CSS), pero requiere levantar un servidor en <code>localhost</code> y abrir el navegador. Viable, pero es una capa extra de complejidad/seguridad que no necesitas para un MVP interno.
⚠ PyQt/PySide	UI más pulida que Tkinter, pero es una dependencia externa pesada y con licencias a revisar (LGPL/GPL). Para un formulario simple es "usar un mazo para clavar un clavo".

Conclusión: empezamos con Tkinter. Si más adelante quieres algo más visual, migrar la lógica (que va toda en `database.py`, separada de la interfaz) a Flask es trivial porque no la tocamos.

2. Diseño de la base de datos

La idea clave de ISA-5.1 que vamos a explotar: **el tag = [Letra Variable][Letra Función]-[Número de lazo]**, y el número de lazo se resuelve dentro de un **rango numérico asignado por área** (así evitamos que Caldera y Evaporación choquen en el mismo número).

Voy a crear el proyecto completo con datos ya precargados con áreas típicas de un ingenio azucarero.

Se ejecutaron 5 comandos

Ahora sí. Empecemos con el esquema de base de datos:

Estructuró la creación de módulo de base de datos con datos iniciales

Ahora el módulo de base de datos con la lógica de negocio (búsqueda de existentes + cálculo del siguiente correlativo):

Módulo de acceso a datos: conexión, inicialización, y lógica de generación de tags según ISA-5.1

Ahora probemos la lógica antes de construir la interfaz gráfica encima:

Validó funcionalidad y planificó construcción de interfaz y base de datos

La lógica funciona exactamente como se pidió: propone el siguiente correlativo dentro del rango del área y bloquea duplicados a nivel de base de datos. Limpio la base de prueba y construyo la interfaz gráfica:

Verificó construcción exitosa de interfaz y procedió limpieza

La GUI arranca sin errores (probada con un display virtual). Limpio la base de prueba y agrego el README con instrucciones:

Equilibró brevedad técnica con explicación necesaria para entrega compleja

Probé todo el flujo end-to-end (generación de correlativos, bloqueo de duplicados, arranque de la interfaz) antes de entregarlo, así que funciona tal cual está.

Cómo quedó resuelto cada requerimiento:

- **Sin conocer ISA-5.1:** el usuario solo elige de 3 listas desplegables (Área → Variable → Función). El sistema arma `PT-201`, `LIC-405`, etc. solo.
- **Sin duplicados:** doble seguro — el cálculo del correlativo revisa el máximo existente, *y además* la columna `tag_completo` tiene una restricción `UNIQUE` en SQLite, así que aunque dos personas intenten crear el mismo tag a la vez, la base de datos rechaza el segundo.
- **Áreas con rango propio:** precargué 10 áreas típicas de un ingenio (Molienda 100-199, Calderas 200-299, Evaporación 400-499, etc.) para que dos áreas nunca choquen en el mismo número de lazo. Si el rango se agota, el sistema avisa en vez de inventar un número fuera de bloque.
- **100% local:** SQLite + Tkinter, ambos parte de la librería estándar de Python — no abre puertos de red ni depende de internet, algo que suele importar en el entorno IT de una planta.

Un par de decisiones que vale la pena que confirmes con tu supervisor:

1. Numeré el correlativo dentro de cada combinación Área+Variable+Función (ej. todos los `PT` de Calderas comparten la secuencia 200, 201, 202...). Si en tu planta el número de lazo se comparte entre instrumentos de un mismo lazo de control (ej. `LT-101`, `LIC-101` y `LV-101` deberían tener el mismo "101"), es un cambio de lógica distinto — avísame y lo ajusto.
2. Las áreas/variables/funciones que precargué son un punto de partida razonable para un ingenio, pero seguro tu supervisor quiere ajustar nombres o rangos exactos — está documentado en el README cómo editarlos.

¿Querés que sigamos con la Fase 2 (pantalla de administración de catálogos, exportar a Excel, o empaquetarlo como `.exe` con PyInstaller para repartirlo sin que la PC destino tenga Python

instalado)?